



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

تمرین سوم فناوری رباتیک

نگارش

محمدرضا منعمیان

اسفند ۱۴۰۴

فهرست مطالب

۱	توسعه نود اختصاصی انتشار نقشه (Custom Map Publisher) و کالیبراسیون آن	۳
۱.۱	هدف پیاده‌سازی	۳
۲.۱	ساختار فایل‌ها و تنظیمات کامپایل (CMakeLists)	۳
۳.۱	منطق پردازش در نود map_publisher.py	۳
۴.۱	هم‌راستاسازی نقشه با شبیه‌ساز (Map Alignment)	۴
۵.۱	اجرا و بصری‌سازی در RViz	۴
۲	پیاده‌سازی الگوریتم مکان‌یابی با فیلتر ذرات (Particle Filter / Monte Carlo Localization)	۵
۱.۲	هدف پیاده‌سازی	۵
۲.۲	مقداردهی اولیه (Global Localization)	۶
۳.۲	مدل حرکت (Motion Model / Prediction Step)	۷
۴.۲	مدل سنسور و وزن‌دهی (Sensor Model & Correction)	۷
۵.۲	بازنمونه‌برداری و مکانیزم بازیابی خطا (Resampling & Error Recovery)	۷
۶.۲	تخمین نهایی مکان ربات	۸
۳	پیاده‌سازی مسیریابی دوبعدی با الگوریتم A* (A* Path Planning)	۹
۱.۳	هدف پیاده‌سازی	۹
۲.۳	دریافت نقشه و مدیریت سیستم مختصات	۱۰
۳.۳	مدل‌سازی گراف حرکتی و تشخیص موانع	۱۰
۴.۳	پیاده‌سازی منطق جستجوی A*	۱۰
۵.۳	راهنمای اجرا و بصری‌سازی تعاملی در RViz	۱۱
۴	پیاده‌سازی سیستم SLAM اختصاصی (Custom 2D SLAM)	۱۲

۱۲	هدف پیاده‌سازی	۱.۴
۱۲	مقداردهی اولیه ساختار نقشه (Map Initialization)	۲.۴
۱۳	تخمین مکان پیوسته (Pose Estimation)	۳.۴
۱۳	به‌روزرسانی نقشه با الگوریتم Raycasting	۴.۴
۱۴	انتشار و رفع مشکل عدم تطابق کیفیت خدمات (QoS)	۵.۴
۱۴	ذخیره‌سازی نقشه روی دیسک (Map Saving Mechanism)	۶.۴
۱۴	راهنمای اجرا و بصری‌سازی در RViz	۷.۴

۱ توسعه نود اختصاصی انتشار نقشه (Custom Map Publisher) و کالبراسیون آن

۱.۱ هدف پیاده‌سازی

در این بخش، هدف این بود که به جای استفاده از پکیج‌های آماده، یک نود اختصاصی در پایتون (`map_publisher.py`) توسعه داده شود تا فایل‌های نقشه محیط (`depot.yaml`) و فایل تصویری متناظر آن را خوانده، پردازش کرده و در قالب استاندارد `nav_msgs/OccupancyGrid` در شبکه ROS 2 منتشر کند. همچنین تنظیم دقیق دستگاه مختصات نقشه با دنیای شبیه‌سازی (Gazebo) از اهداف کلیدی این بخش بود.

۲.۱ ساختار فایل‌ها و تنظیمات کامپایل (CMakeLists)

برای این که پکیج به درستی کار کند، ابتدا ساختار فایل‌ها مرتب شد:

- یک پوشه به نام `maps` در مسیر پکیج ایجاد شد و فایل‌های نقشه شامل تصویر و `.yaml` درون آن قرار گرفت.
- فایل `CMakeLists.txt` به‌روزرسانی شد. با استفاده از دستورات (`install(DIRECTORY ...)` و (`install(PROGRAMS ...)`، به کامپایلر (`colcon`) دستور داده شد تا پوشه `maps` و اسکریپت `map_publisher.py` را در زمان بیلد شدن به مسیر `install` (فضای اجرایی) منتقل کند تا نود بتواند در زمان اجرا به آن‌ها دسترسی داشته باشد.

۳.۱ منطق پردازش در نود `map_publisher.py`

نود نوشته شده چند عملیات حیاتی را برای تبدیل عکس به نقشه قابل فهم برای ربات انجام می‌دهد:

- **تنظیمات QoS:** برای تاییک `map` / از سیاست `DurabilityPolicy.TRANSIENT_LOCAL` استفاده شد. این یک تنظیم بسیار مهم است و تضمین می‌کند نودهایی که دیرتر روشن می‌شوند (مثل RViz یا نودهای مسیریابی)، همچنان بتوانند آخرین نقشه منتشر شده را دریافت کنند.

- پردازش تصویر (Image Processing): فایل تصویر با کتابخانه PIL به صورت سیاه و سفید (Grayscale) خوانده شد. از آنجایی که مبدأ مختصات تصاویر در برنامه نویسی معمولاً بالا-چپ است، اما در ROS 2 مبدأ نقشه پایین-چپ در نظر گرفته می شود، از تابع `np.flipud` برای قرینه کردن تصویر در راستای عمودی استفاده شد.
- تخصیص مقادیر اشغال شدگی (Occupancy Values): با استفاده از آستانه های تعریف شده در فایل YAML (`occupied_thresh` و `free_thresh`)، مقادیر پیکسل ها به درصدهای احتمال تبدیل شدند. پیکسل های تیره (دیوارها) مقدار ۱۰۰، پیکسل های روشن (فضای خالی) مقدار ۰، و سایر نقاط مقدار ۱- (فضای ناشناخته) گرفتند.

۴.۱ هم راستاسازی نقشه با شبیه ساز (Map Alignment)

برای اینکه نقشه تولید شده دقیقاً زیر پای ربات در شبیه ساز Gazebo قرار بگیرد، نیاز به کالیبراسیون مبدأ (Origin) بود.

- با بررسی محیط سه بعدی Gazebo، مختصات گوشه پایین-چپ دیوارها استخراج شد ($X = -۱۵/۳۵$ و $Y = -۷/۸۰$).
- این مقادیر در فایل `depot.yaml` به عنوان پارامتر `origin` تنظیم شدند تا مرکز نقشه با مختصات جهانی شبیه ساز همگام شود. نود `map_publisher` با خواندن این مقادیر، پارامتر `msg.info.origin.position` را به درستی مقداردهی می کند.

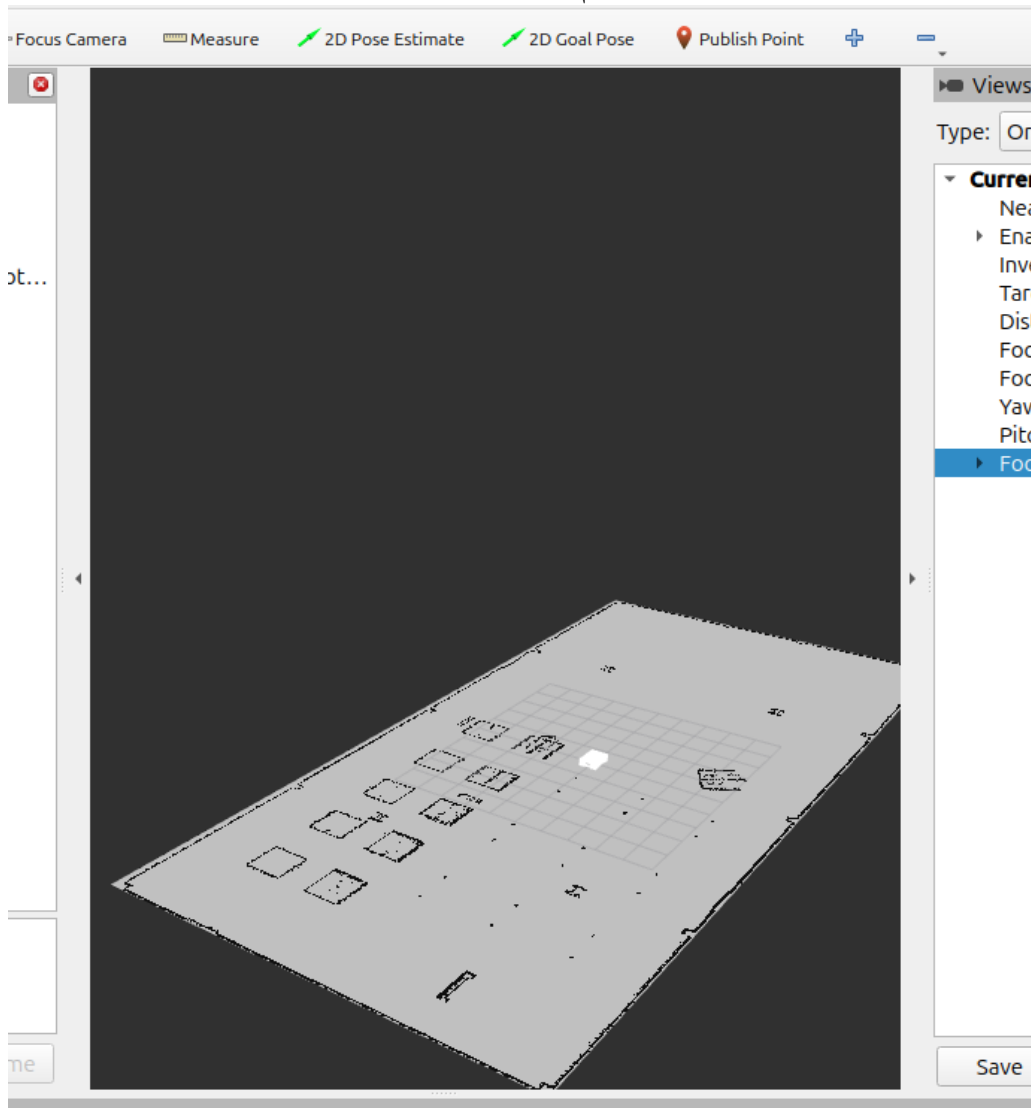
۵.۱ اجرا و بصری سازی در RViz

پس از بیلد کردن پروژه، نود اختصاصی اجرا شد. برای مشاهده نتیجه در نرم افزار RViz:

- پارامتر `Fixed Frame` در تنظیمات سراسری بر روی `map` تنظیم شد.
- با اضافه کردن ابزار نمایش `Map` و اتصال آن به تاپیک `/map`، نقشه پردازش شده با موفقیت به نمایش درآمد.

بررسی ها نشان داد که به لطف تنظیم دقیق `Origin`، دیوارهای نقشه کاملاً با دنیای فیزیکی و داده های سنسوری ربات منطبق هستند.

در زیر عکسی از نقشه را مشاهده میکنیم :



شکل ۱: لود شدن نقشه بر rviz

۲ پیاده‌سازی الگوریتم مکان‌یابی با فیلتر ذرات (Particle Filter / Monte Carlo Localization)

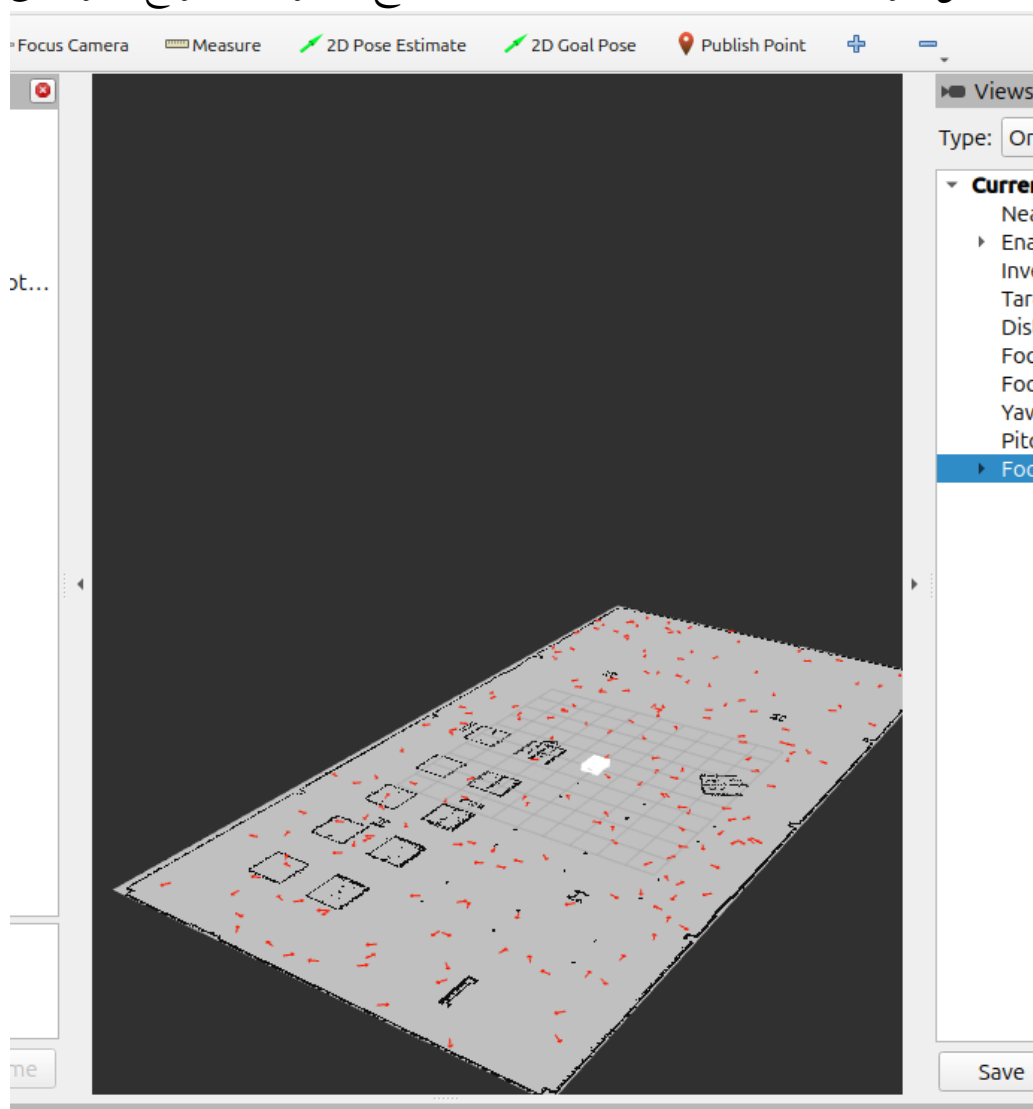
۱.۲ هدف پیاده‌سازی

در این بخش، یک نود اختصاصی برای مکان‌یابی ربات بر پایه الگوریتم فیلتر ذرات (MCL) از پایه (From Scratch) در پایتون توسعه داده شد. هدف این الگوریتم، تخمین مکان ربات در نقشه از

پیش‌ساخته شده، با ترکیب داده‌های اودومتری (حرکت کور) و لیزر اسکنر (مشاهده محیط) بود. برای ساده‌سازی فرآیند تست و توسعه در این مرحله، درخت تبدیل مختصات (TF) بین فریم‌های map و odom به صورت استاتیک (Static Transform) مقداردهی و منتشر شد.

۲.۲ مقداردهی اولیه (Global Localization)

زمانی که نود `particle_filter` نقشه را دریافت می‌کند، ربات هیچ ایده‌ای از مکان خود ندارد. بنابراین، تابع `initialize_particles` اجرا شده و تعداد ۲۰۰ ذره (Particle) را به صورت کاملاً تصادفی در نقشه پخش می‌کند. در این مرحله یک شرط منطقی (`self.map_data == 0`) تعبیه شده است تا اطمینان حاصل شود ذرات تنها در فضاهای آزاد نقشه (خارج از دیوارها و موانع) متولد می‌شوند.



شکل ۲: مقداردهی اولیه پارتیکل‌ها

۳.۲ مدل حرکت (Motion Model / Prediction Step)

با دریافت داده‌ها از تایپیک اودومتری (`/ekf_diff_imu/odom`)، میزان جابجایی و چرخش ربات $(\Delta x, \Delta y, \Delta \theta)$ نسبت به لحظه قبل محاسبه می‌شود. سپس این جابجایی به تک‌تک ۲۰۰ ذره اعمال می‌گردد. از آنجایی که در دنیای واقعی حرکت ربات با لغزش و خطا همراه است، در کد مقادیر نویز گوسی (Gaussian Noise) متناسب با میزان مسافت طی شده به حرکت ذرات اضافه گردید تا عدم قطعیت سنسور اودومتری مدل‌سازی شود.

۴.۲ مدل سنسور و وزن‌دهی (Sensor Model & Correction)

وظیفه تطبیق مشاهدات ربات با نقشه در تابع `scan_callback` انجام می‌شود. برای جلوگیری از اضافه‌بار پردازشی، داده‌های لیزر (`/scan`) با گام ۱۰ (Downsampling) کاهش یافتند. برای هر ذره، فرض شد که اگر ربات در مکان آن ذره قرار داشت، پرتوهای لیزر به کجا برخورد می‌کردند. نقاط برخورد فرضی با نقشه تطبیق داده شدند؛ اگر نقطه برخورد روی دیوار (`map_data > 50`) قرار می‌گرفت، ذره امتیاز مثبت دریافت می‌کرد. در نهایت، امتیازات به توان ۲ رسیده و به عنوان وزن (Weight) به ذرات اختصاص یافتند تا ذرات نزدیک‌تر به واقعیت، وزن بسیار بیشتری پیدا کنند.

۵.۲ بازنمونه‌برداری و مکانیزم بازیابی خطا (Resampling & Error Recovery)

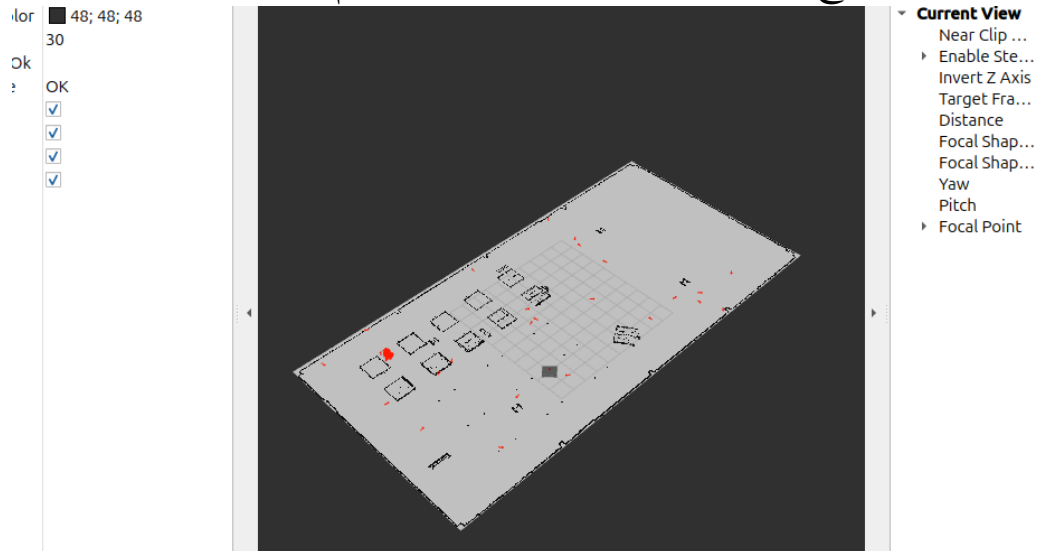
یکی از مهم‌ترین چالش‌های پارسیکل فیلتر، مشکل قفل شدن ذرات در یک مکان اشتباه (Kidnapped Robot Problem) است. برای حل این مشکل، شاخص تنوع ذرات (N_{eff}) محاسبه شد. در صورت کاهش تنوع، فرآیند ریسمپلینگ با یک رویکرد هیبریدی انجام گردید:

- **بقای اصلح (۹۰ درصد):** ۹۰٪ ذرات جدید از بین ذرات موجود و بر اساس وزن آن‌ها انتخاب و با اندکی نویز تکثیر شدند.
- **تزریق ذرات تصادفی (۱۰ درصد):** برای جلوگیری از همگرایی اشتباه و اصلاح خطاهای احتمالی در طول زمان، همواره ۱۰٪ از ظرفیت به ذراتی اختصاص یافت که به صورت کاملاً تصادفی در فضاهای آزاد نقشه (Random Injection) متولد می‌شوند. این ایده باعث می‌شود اگر ربات مکان خود را گم کرد، ذرات تصادفی بتوانند مکان جدید را به سرعت پیدا کرده و سایر ذرات را به آن سمت جذب کنند.

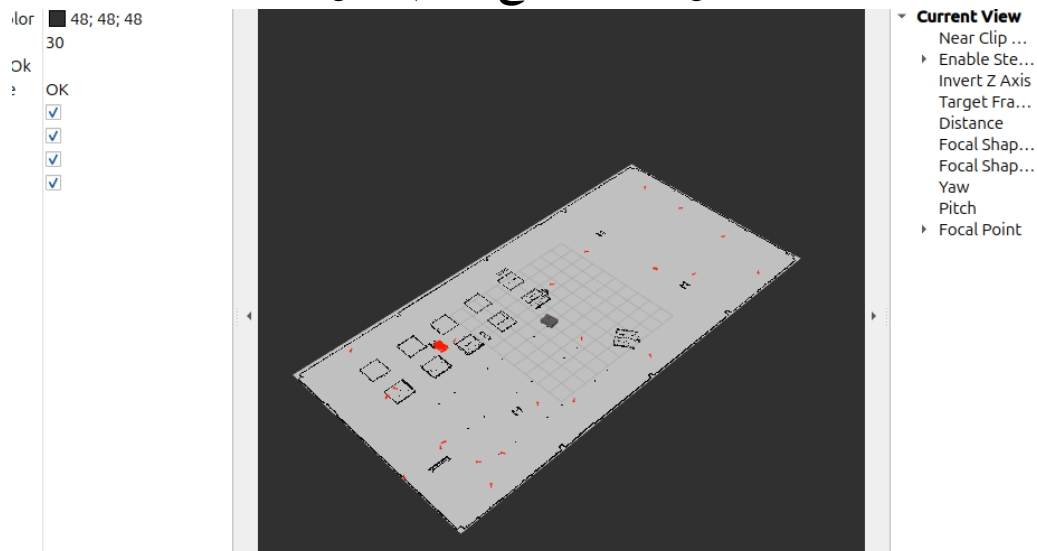
۶.۲ تخمین نهایی مکان ربات

در انتها، پس از فیلتر شدن ذرات ضعیف و تجمع ذرات قوی در اطراف مکان واقعی ربات، تابع `estimate_and_publish_pose` میانگین مختصات (X, Y) و میانگین زاویه (Yaw) ذرات را محاسبه کرده و به عنوان مکان قطعی و تخمین زده شده ربات در تاپیک `/amcl_pose` منتشر می‌کند. نتیجه این فرآیند، مکان‌یابی بسیار دقیق ربات همزمان با حرکت در محیط شبیه‌ساز بود.

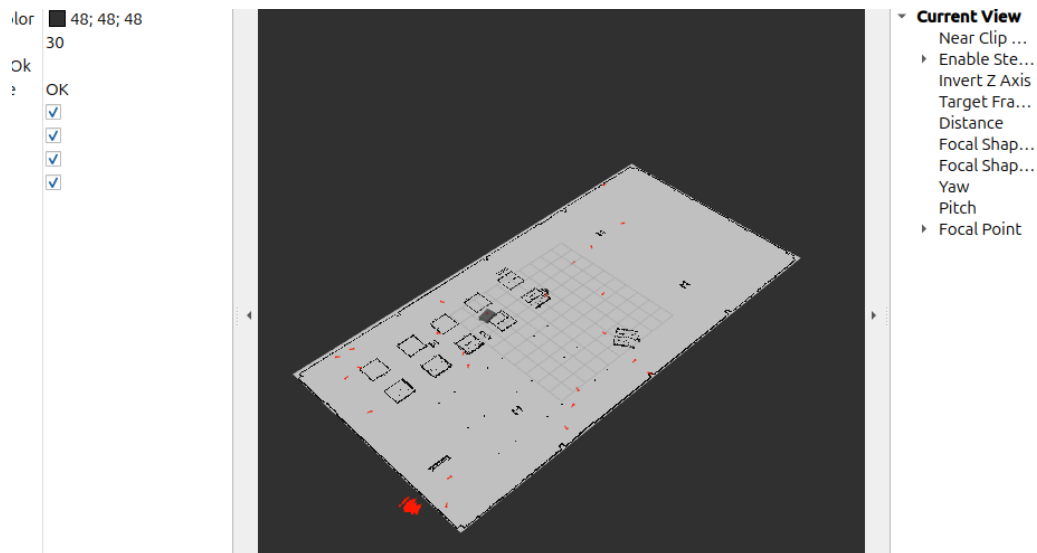
در ادامه روند کانورج کردن ذرات پارتیکل را مشاهده می‌کنیم:



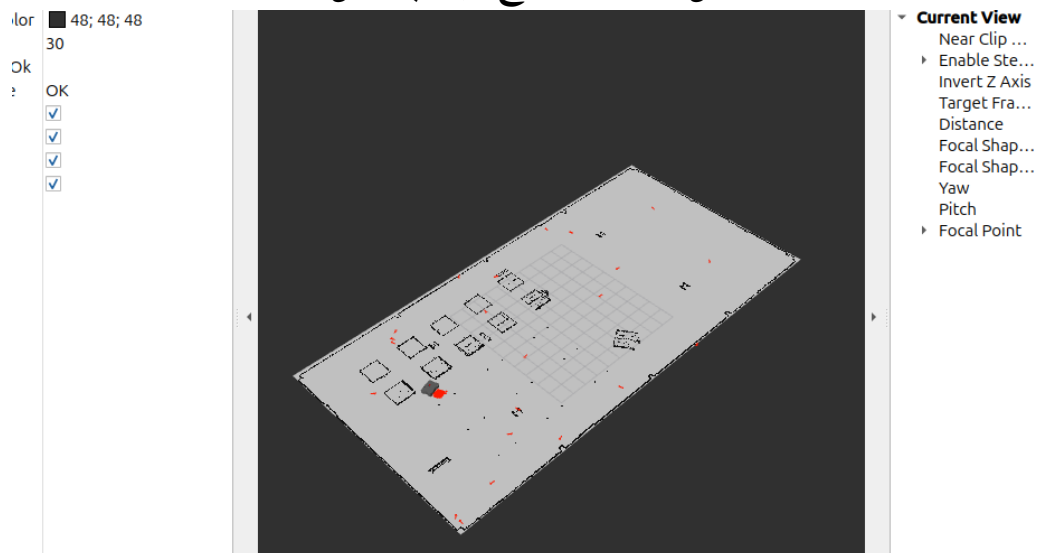
شکل ۳: روند کانورج کردن پارتیکل‌ها



شکل ۴: روند کانورج کردن پارتیکل‌ها



شکل ۵: روند کانورج کردن پارتیکل ها



شکل ۶: روند کانورج کردن پارتیکل ها

۳ پیاده‌سازی مسیریابی دوبعدی با الگوریتم A* (A* Path Planning)

۱.۳ هدف پیاده‌سازی

هدف از این بخش، توسعه یک نود مسیریاب تعاملی (AStarInteractive) بر پایه نقشه شبکه‌ای (Grid-based) بود. این نود موظف است با دریافت نقشه محیط (OccupancyGrid) و گرفتن نقاط مبدأ و مقصد به صورت تعاملی از کاربر، کوتاه‌ترین و بهینه‌ترین مسیر را با دور زدن موانع محاسبه کرده و در قالب یک پیام استاندارد (nav_msgs/Path) منتشر کند.

۲.۳ دریافت نقشه و مدیریت سیستم مختصات

برای اطمینان از دریافت صحیح نقشه، از تنظیمات QoSProfile با ویژگی TRANSIENT_LOCAL استفاده شد تا حتی در صورت اجرای دیر هنگام نود مسیریاب، آخرین نقشه موجود در شبکه دریافت شود. از آنجا که محاسبات الگوریتم A^* بر روی ماتریس‌ها (پیکسل/سلول) انجام می‌شود اما نقاط دریافتی از RViz بر اساس واحد متریک (سیستم مختصات جهانی X و Y) هستند، دو تابع کلیدی world_to_grid و grid_to_world توسعه یافتند. این توابع با استفاده از پارامترهای resolution و مبدأ نقشه (origin)، عمل مقیاس‌بندی و تبدیل مختصات پیوسته به گسسته (و بالعکس) را با دقت بالا انجام می‌دهند.

۳.۳ مدل‌سازی گراف حرکتی و تشخیص موانع

برای حرکت در نقشه، یک گراف با اتصال ۸-گانه (8-way connectivity) در نظر گرفته شد؛ به طوری که هزینه حرکت در جهات اصلی برابر با ۰.۱ و در جهات قطری برابر با ۱/۴۱۴ (رادیكال ۲) لحاظ گردید (تابع get_neighbors). همچنین تابع is_valid مسئولیت بررسی برخورد با موانع را بر عهده دارد. در این تابع، سلول‌هایی که مقدار آن‌ها در نقشه بیشتر از ۵۰ (احتمال بالای وجود دیوار) یا برابر با -۱ (فضای ناشناخته) باشد، به عنوان مسیرهای غیرمجاز مسدود می‌شوند.

۴.۳ پیاده‌سازی منطق جستجوی A^*

هسته اصلی مسیریابی در تابع a_star با بهره‌گیری از رابطه کلاسیک $f(n) = g(n) + h(n)$ پیاده‌سازی شد:

- هزینه طی شده $g(n)$: نشان‌دهنده فاصله واقعی از نقطه شروع تا گره فعلی است.
- تابع هیوریستیک $h(n)$: برای تخمین فاصله تا هدف، از فاصله اقلیدسی (Euclidean Distance) با استفاده از تابع math.hypot استفاده شد.

برای بهینه‌سازی سرعت جستجو و جلوگیری از پیمایش کل نقشه، از ساختمان داده صف اولویت‌دار (Priority Queue) با استفاده از کتابخانه heapq پایتون استفاده گردید. این امر باعث می‌شود الگوریتم همواره گره‌هایی را که کمترین مقدار f-score را دارند، در اولویت بررسی قرار دهد. در

نهایت، پس از رسیدن به هدف، مسیر طی شده از طریق دیکشنری `came_from` بازسازی شده و برای انتشار آماده می‌شود.

۵.۳ راهنمای اجرا و بصری‌سازی تعاملی در RViz

کامپایل و اجرای نود:

ابتدا فضای کاری (Workspace) با دستور `colcon build` کامپایل شده و سپس نود مربوطه اجرا می‌گردد:

```
1 ros2 run robot_description a_star_interactive.py
```

تنظیمات RViz:

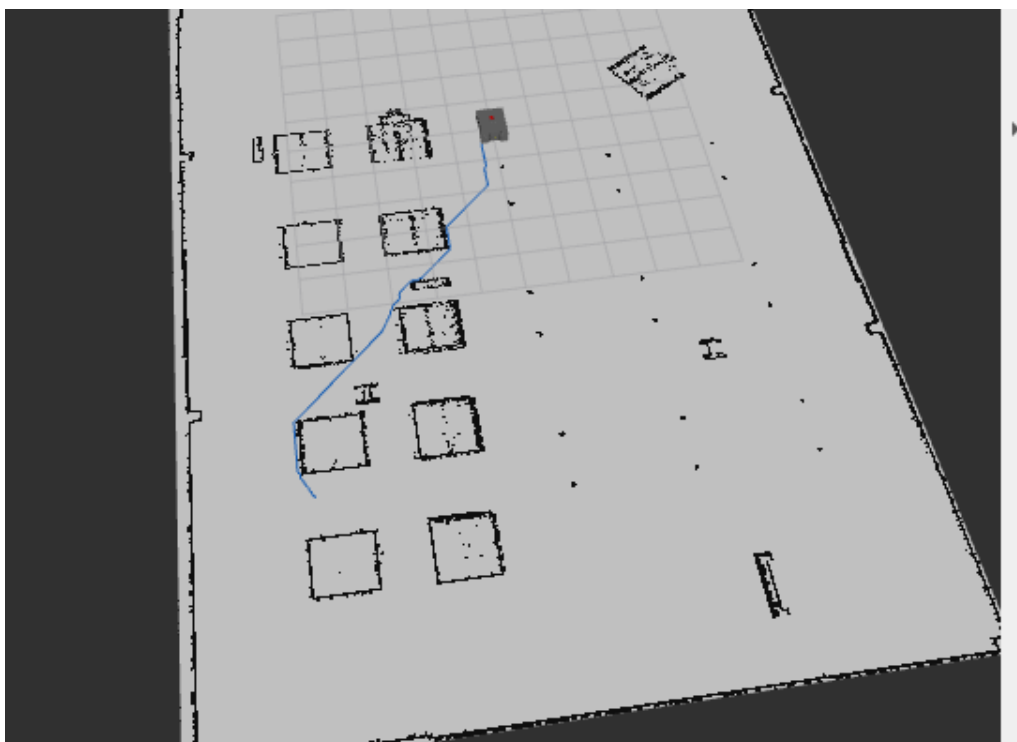
در محیط RViz، پس از بارگذاری نقشه (Map)، با کلیک بر روی دکمه `Add` یک نمایشگر از نوع `Path` به محیط اضافه می‌شود. تاپیک این نمایشگر باید بر روی `/plan` تنظیم گردد تا مسیر خروجی را دریافت کند. (پیشنهاد می‌شود رنگ خط مسیر بر روی سبز یا آبی روشن تنظیم شود تا کنتراست مناسبی با نقشه داشته باشد).

تعیین نقاط و رسم مسیر:

- **نقطه شروع (Start):** کاربر از نوار ابزار بالای RViz دکمه `2D Pose Estimate` را انتخاب کرده و بر روی نقطه‌ای آزاد از نقشه کلیک می‌کند. این کار پیام را به تاپیک `/initialpose` ارسال می‌کند.

- **نقطه هدف (Goal):** بلافاصله پس از آن، کاربر با انتخاب دکمه `2D Goal Pose` و کلیک بر روی نقطه مقصد (مثلاً پشت یک مانع)، پیام را به تاپیک `/goal_pose` می‌فرستد.

نتیجه: نود `a_star_interactive` در کسری از ثانیه محاسبات را انجام داده و مسیر بهینه به صورت یک خط پیوسته روی نقشه رسم می‌شود.



شکل ۷: مسیریابی از طریق a^*

۴ پیاده‌سازی سیستم SLAM اختصاصی (Custom 2D SLAM)

۱.۴ هدف پیاده‌سازی

بر اساس نیازمندی‌های این بخش (Bonus)، هدف طراحی و پیاده‌سازی یک سیستم SLAM دوبعدی از پایه (From Scratch) بدون استفاده از پکیج‌های آماده و استاندارد ROS (مانند gmapping، Cartographer و غیره) بود. این نود (`custom_slam.py`) موظف است با دریافت همزمان داده‌های اودومتری و سنسور لیزر (LiDAR)، مکان ربات را تخمین زده و یک نقشه شبکه‌ای (Occupancy Grid) از محیط ناشناخته را به صورت تدریجی (Incremental) بسازد و در نهایت روی دیسک ذخیره کند.

۲.۴ مقداردهی اولیه ساختار نقشه (Map Initialization)

در ابتدای کار، یک ماتریس دوبعدی (Numpy Array) با ابعاد ۸۰۰ در ۸۰۰ پیکسل (معادل یک محیط ۴۰ در ۴۰ متر با رزولوشن ۰.۵۰ متر بر پیکسل) ایجاد شد. تمام درایه‌های این ماتریس با مقدار

۱- مقداردهی اولیه شدند که در استاندارد ROS نشان‌دهنده «فضای ناشناخته» (Unknown Space) است. هدف الگوریتم، تبدیل این فضاها به عدد ۰ (فضای آزاد/امن) و عدد ۱۰۰ (دیوار/مانع) بر اساس مشاهدات لیزر است.

۳.۴ تخمین مکان پیوسته (Pose Estimation)

برای پیگیری مکان ربات در محیط، نود به تاپیک `/ekf_diff_imu/odom` متصل شد. تابع `odom_callback` به صورت پیوسته موقعیت خطی (X, Y) و جهت‌گیری (Yaw) ربات را دریافت و به‌روزرسانی می‌کند. این داده‌ها به عنوان نقطه مبدأ برای رسم پرتوهای لیزر در هر لحظه استفاده می‌شوند.

۴.۴ به‌روزرسانی نقشه با الگوریتم Raycasting

قلب تپنده این سیستم SLAM، در تابع `scan_callback` و پردازش داده‌های لیزر اسکنر نهفته است. برای نگاشت مشاهدات لیزر بر روی ماتریس نقشه، از الگوریتم گرافیکی برسنهام (Bresenham's Line Algorithm) استفاده شد. فرآیند به این شکل است:

۱. ابتدا نقطه برخورد هر پرتو لیزر با مانع در مختصات جهانی (با استفاده از توابع مثلثاتی سینوس و کسینوس و طول پرتو) محاسبه می‌شود.

۲. مختصات جهانی ربات و نقطه برخورد، توسط تابع `world_to_grid` به مختصات پیکسلی (ایندکس‌های ماتریس) تبدیل می‌گردند.

۳. الگوریتم Bresenham تمام پیکسل‌های قرار گرفته روی خط واصل بین ربات و نقطه برخورد را استخراج می‌کند.

۴. پیکسل انتهایی (نقطه برخورد) مقدار ۱۰۰ (دیوار) گرفته و تمامی پیکسل‌های میانی که لیزر از آن‌ها عبور کرده است، مقدار ۰ (فضای خالی) می‌گیرند.

۵. این فرآیند به صورت تدریجی، تاریکی‌های نقشه (۱-) را کنار زده و محیط را کشف می‌کند.

۵.۴ انتشار و رفع مشکل عدم تطابق کیفیت خدمات (QoS)

ماتریس تولید شده در هر گام به فرمت پیام `nav_msgs/OccupancyGrid` تبدیل می‌شود. یکی از چالش‌های پیاده‌سازی در این بخش، عدم نمایش نقشه در RViz به دلیل تداخل سیاست‌های QoS بود. برای رفع این مشکل، پابلیشر نقشه با پروفایل `DurabilityPolicy.TRANSIENT_LOCAL` تنظیم گردید تا نقشه در حافظه شبکه باقی بماند و گیرنده‌ها (مانند RViz) دچار خطای `Incompatible QoS` نشوند.

۶.۴ ذخیره‌سازی نقشه روی دیسک (Map Saving Mechanism)

برای ذخیره نقشه تولید شده (طبق خواسته سوال)، یک سرویس اختصاصی (Service) با نام `/save_map` از نوع `std_srvs/Empty` توسعه یافت. با فراخوانی این سرویس، تابع `save_map_callback` ماتریس نقشه را خوانده و مقادیر ۱۰۰، ۰ و ۱ را به ترتیب به رنگ‌های سیاه (۰)، سفید (۲۵۵) و خاکستری (۲۰۵) تبدیل می‌کند. خروجی به صورت یک فایل تصویر با فرمت استاندارد `pgm` و یک فایل متادیتا با فرمت `yaml` مستقیماً روی دیسک ذخیره می‌شود.

۷.۴ راهنمای اجرا و بصری‌سازی در RViz

برای اجرای سیستم نقشه‌برداری همزمان و استخراج نقشه نهایی، مراحل زیر در محیط ROS 2 انجام می‌پذیرد:

اجرای نودها:

ابتدا شبیه‌ساز Gazebo اجرا شده و برای برقراری ارتباط بین فریم‌های محلی، یک تبدیل TF استاتیک بین `map` و `odom` منتشر می‌شود. سپس نود SLAM با دستور زیر اجرا می‌گردد:

```
1 ros2 run robot_description custom_slam.py
```

بصری‌سازی همزمان (Real-time Visualization):

در نرم‌افزار RViz، پارامتر `Fixed Frame` روی `map` تنظیم می‌شود. با اضافه کردن نمایشگر `Map` (متصل به تاپیک `/map`) و `LaserScan` (متصل به تاپیک `/scan`)، کاربر می‌تواند پرتوهای لیزر را

ببیند.

اکتشاف (Exploration):

ربات با استفاده از گره Teleop (کنترل کیبورد) در محیط Gazebo حرکت داده می‌شود. همزمان با حرکت، نقشه در RViz پیکسل به پیکسل رسم شده و دیوارهای محیط نمایان می‌شوند.

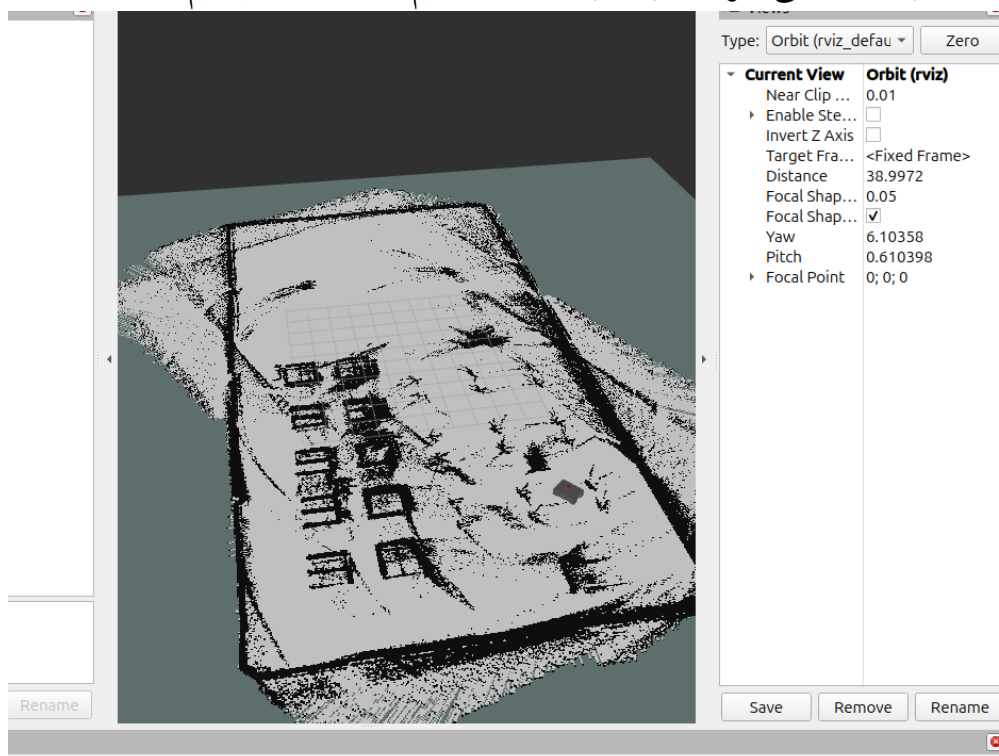
ذخیره نقشه (Save Map):

پس از تکمیل اسکن محیط، کاربر با اجرای دستور زیر در یک ترمینال مجزا، سرویس ذخیره‌سازی را فراخوانی می‌کند:

```
1 ros2 service call /save_map std_srvs/srv/Empty
```

با اجرای این دستور، دو فایل my_custom_map.pgm و my_custom_map.yaml در پوشه جاری ساخته شده که نشان‌دهنده اتمام موفقیت‌آمیز مأموریت نقشه‌برداری است.

نقشه ای که بعد از مدتی حرکت ربات به دست آوردیم را مشاهده میکنیم :



شکل ۸: slam